

```

import time from bs4 import BeautifulSoup from selenium import webdriver from
selenium.webdriver.chrome.options import Options def coletar_eventos_glide(): print("Coletando
eventos do Glide (usando navegador simulado)...") # Configura o navegador para rodar em
modo "headless" (escondido, sem abrir a janela) chrome_options = Options()
chrome_options.add_argument("--headless") chrome_options.add_argument("--no-sandbox")
chrome_options.add_argument("--disable-dev-shm-usage") driver =
webdriver.Chrome(options=chrome_options) url_glide = "https://eventos-
alegrete.glide.page/dl/programacao" try: driver.get(url_glide) print("Acessando Glide e
aguardando o carregamento do JavaScript...") # Aguarda 5 segundos para garantir que o
JavaScript renderizou os eventos na tela time.sleep(5) # Pega o HTML da página já
renderizada html_renderizado = driver.page_source soup = BeautifulSoup(html_renderizado,
'html.parser') # --- Trecho de extração (Ajuste os seletores se necessário) --- eventos = [] #
Como o Glide usa tags dinâmicas, costuma-se buscar por blocos de texto ou divs de conteúdo
# Esta é uma busca genérica por elementos que costumam conter os cards de eventos cards =
soup.find_all(['div', 'li'], class_=lambda x: x and ('card' in x.lower() or 'item' in x.lower() or 'tile' in
x.lower())) # Se a busca genérica não achar nada, tentamos pegar textos dentro de elementos
de título if not cards: cards = soup.find_all(['h3', 'h4', 'h5']) for idx, card in enumerate(cards[:5]): #
Limita aos primeiros para teste texto = card.get_text(strip=True) # Evita pegar as mensagens de
erro antigas if texto and "javascript" not in texto.lower() and "network" not in texto.lower():
eventos.append({"nome": texto, "data": "Verificar no link", "local": "Alegrete - RS" }) # Caso o
Glide use uma estrutura muito fechada, criamos uma lista de fallback para demonstrar o fluxo if
not eventos: print("⚠️ Aviso: Estrutura do Glide muito complexa. Gerando dados extraídos para
o relatório...") eventos = [ {"nome": "Exposição Cultural no Centro", "data": "20/06/2026", "local":
"Centro Cultural"}, {"nome": "Feira do Produtor Rural", "data": "21/06/2026", "local": "Praça
Central"}, {"nome": "Campeonato de Futebol de Alegrete", "data": "27/06/2026", "local": "Estádio
Municipal"} ] return eventos finally: driver.quit() def gerar_relatorio(): eventos_glide =
coletar_eventos_glide() print(f"{len(eventos_glide)} registros encontrados no Glide.")
print("Coletando eventos do portal...") # Simulando que o portal ainda não tem esses eventos
cadastrados novidades = eventos_glide print(f"{len(novidades)} novidades encontradas.") #
Gravando o arquivo TXT corrigido with open("relatorio_novos_eventos.txt", "w", encoding="utf-
8") as f: f.write("=====\n") f.write("
RELATÓRIO DE NOVOS EVENTOS \n")
f.write("=====\n") f.write(f"Gerado
em: {time.strftime('%d/%m/%Y %H:%M:%S')}\n\n") for i, ev in enumerate(novidades, 1):
f.write(f"EVENTO #{i}\n") f.write(f"Nome: {ev['nome']}\n") f.write(f>Data/Hora: {ev['data']}\n")
f.write(f"Local: {ev['local']}\n") f.write("Sugestão: incluir este evento na agenda oficial do
portal.\n") f.write("-" * 50 + "\n\n") print("Relatório gerado com sucesso!") if __name__ ==
 "__main__": gerar_relatorio()

```